

Processus de Développement

Dr. Kungne Willy

Enseignant-Chercheur à Ecole Nationale Supérieure Polytechnique de Yaoundé.

Plan

- Facteurs de Qualités
-
- Processus de développements
-
- Activités
-
- Méthodes de développements

Les facteurs de qualité

- Les bogues surviennent quand le logiciel ne correspond pas au besoin.
 - Pour améliorer la qualité du logiciel, il y a des questions à se poser très en amont.
 -
 - D'où il faut attendre les qualités.

•

Les facteurs de qualité

- **Portabilité :**

- Utilisation d'un langage standardisé
- Indépendance du matériel
- Indépendance du système d'exploitation

- **Validité :** aptitude d'un produit logiciel à remplir exactement ses fonctions, définies par le cahier des charges et les spécifications.

- **Fiabilité** (ou robustesse) : aptitude d'un produit logiciel à fonctionner dans des conditions anormales.

- **Extensibilité :** facilité avec laquelle un logiciel se prête à une modification ou à une extension des fonctions qui lui sont demandées.

- **Réutilisabilité :** aptitude d'un logiciel à être réutilisé, en tout ou en partie, dans de nouvelles applications.

- **Compatibilité :** facilité avec laquelle un logiciel peut être combiné avec d'autres logiciels.

-

Les facteurs de qualité

- **Efficacité** : Utilisation optimales des ressources matérielles.
- **Portabilité** : facilité avec laquelle un logiciel peut être transférée sous différents environnements matériels et logiciels.
- **Vérifiabilité** : facilité de préparation des procédures de test.
- **Intégrité** : aptitude d'un logiciel à protéger son code et ses données contre des accès non autorisés.
- **Facilité d'emploi** : facilité d'apprentissage, d'utilisation, de préparation des données, d'interprétation des erreurs et de rattrapage en cas d'erreur d'utilisation.

Les facteurs de qualité : Problème

- Ces facteurs sont parfois contradictoires, le choix des compromis doit s'effectuer en fonction du contexte.
- Par exemple, la facilité d'emploi et la fiabilité peuvent être contradictoire.
 - Dans une application du type « traitement de texte » c'est le premier de ces deux facteurs qui sera favorisé.
 - Dans une application de pilotage d'usine c'est le deuxième de ces deux facteurs qui sera favorisé.
- Trouver le point optimal, car l'amélioration d'une qualité influence automatiquement les autres
- Le coût peut croître exponentiellement en fonction de certains besoins de qualité

- Pour obtenir ces qualités, il faut **s'imposer un processus** pour la production des logiciels afin que le logiciel produit soit en **adéquation** avec les besoins utilisateurs.
-
- Le **processus de développement** va aussi nous aider pour maîtriser les **coûts** et les **délais**

Processus de developpement

- L'économie mondiale actuellement est dépendante du logiciel. De plus en plus de systèmes sont contrôlés par le logiciel
 - Banque
 - Distributeur automatique
 - Avions
 - Navettes spatiales
 - Téléphonie etc ...
- Le concepteur de logiciels doit d'abord étudier les besoins de clients afin de voir ce qu'il peut apporter en matière d'adaptation informatique. Ensuite, il réalise un logiciel qui va combler les manques qui ont été repérés

Processus de développement : c'est quoi ?

- Le développement et l'évolution d'un logiciel doit être vu comme un processus.
- **Processus :**
 - Ensemble d'activités coordonnées et régulées, en partie ordonnées, dont le but est de créer un produit.
 - Il décrit la séquence des activités d'ingénierie nécessaires pour transformer les besoins et les exigences exprimés par le client en un produit
- **Processus de développement**
 - L'ordre et la manière d'enchaîner les étapes d'un développement

Processus de développement : c'est quoi ?

- Il fournit un cadre méthodologique dans la mesure où il définit:
 - Des activités
 - Le cadre d'affectation des ressources
 - Les ressources humaines
 - Les outils
 - Le temps
 - Le budget
 - Un schéma d'ordonnancement et de contrôle des tâches
-
- Un **processus** décrit 2 choses importantes :
 - Les activités (étapes) (QUOI ?)
 - L'enchaînement des activités (QUAND ?)

Une Activité c'est quoi ?

Les activités d'un processus sont souvent décrites en termes de :

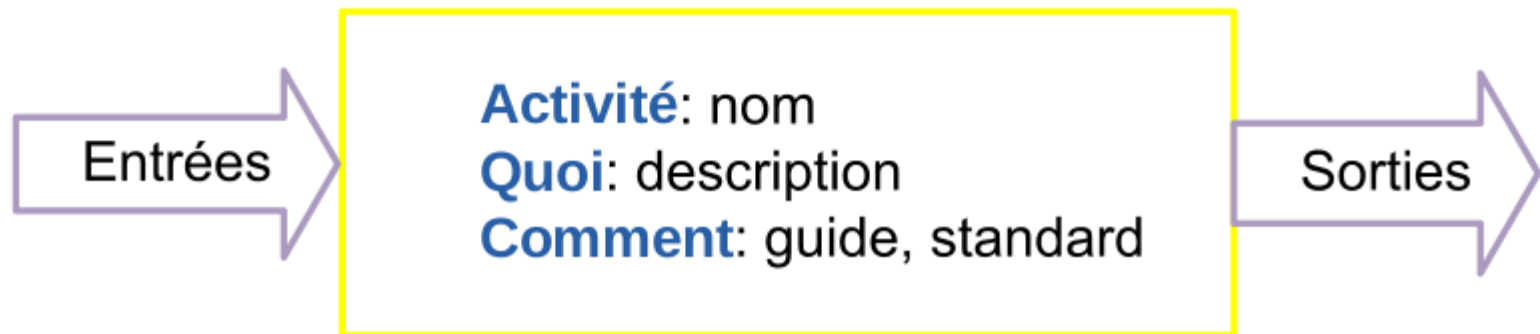
Entrées de l'activité (matière première)

Sorties de l'activité (résultat)

Intervenants et rôles (qui ?)

Description de l'activité (quoi - quel est le problème à traiter ?)

Standards, guides, « best practices » à appliquer (comment ?)



Activités : les classiques

Spécifier <i>Qu'est-ce que le logiciel doit faire?</i> <i>Comment s'assurer qu'il le fait?</i> <i>Comment s'assurer qu'on développe le bon logiciel?</i>	Valider <i>Le logiciel fait-il ce qu'il doit faire?</i> <i>Développe t on le bon logiciel?</i>
Concevoir <i>Comment organiser le logiciel pour qu'il fasse ce qu'il doit faire?</i> <i>Quelles choix techniques faut-il faire pour que le logiciel fasse ce qu'il doit faire?</i> <i>Comment s'assurer que le logiciel est organisé et construit de manière à faire ce qu'il doit faire?</i>	Intégrer <i>Le logiciel est-il organisé et construit de manière à faire ce qu'il doit faire?</i>
Coder <i>Comment traduit-on cette organisation en code source?</i>	Tester unitairement <i>Le code source est-il bien écrit?</i>

Activités : *Faisabilité*

- Faisabilité

- Questions

- Pourquoi développer le logiciel ?
 - Y a-t-il de meilleures alternatives ?
 - Comment procéder pour faire ce développement ?
 - Y a-t-il un marché pour le logiciel ?
 - Quels moyens faut-il mettre en oeuvre ? A-t-on le budget, le personnel, le matériel nécessaires ?

Activités : *Faisabilité*

Activité	Étude préalable
Description	Etudier la faisabilité du projet, ses contraintes techniques (coût, temps, qualité) et les alternatives possibles
Entrées	Problème à résoudre, objectifs à atteindre
Sorties	OUI (la décision est prise de réaliser le logiciel) ou NON (le projet est abandonné)



Activités : *Spécification*

Activité	Spécifier
Description	Décrire ce que doit faire le logiciel (comportement en boîte-noire). Décrire comment vérifier en boîte-noire que le logiciel fait bien ce qui est exigé.
Entrées	Client qui a une idée de ce qu'il veut. Exigences, désir, besoins. . . concernant le système permettant de résoudre le problème.
Sorties	Cahier des charges du logiciel (ou spécification du logiciel). Des procédures de validation. Version provisoire des manuels d'utilisation et d'exploitation du logiciel

Activités : *Spécification*

Activité	Spécifier
Description	Décrire ce que doit faire le logiciel (comportement en boîte-noire). Décrire comment vérifier en boîte-noire que le logiciel fait bien ce qui est exigé.
Entrées	Client qui a une idée de ce qu'il veut. Exigences, désir, besoins. . . concernant le système permettant de résoudre le problème.
Sorties	Cahier des charges du logiciel (ou spécification du logiciel). Des procédures de validation. Version provisoire des manuels d'utilisation et d'exploitation du logiciel

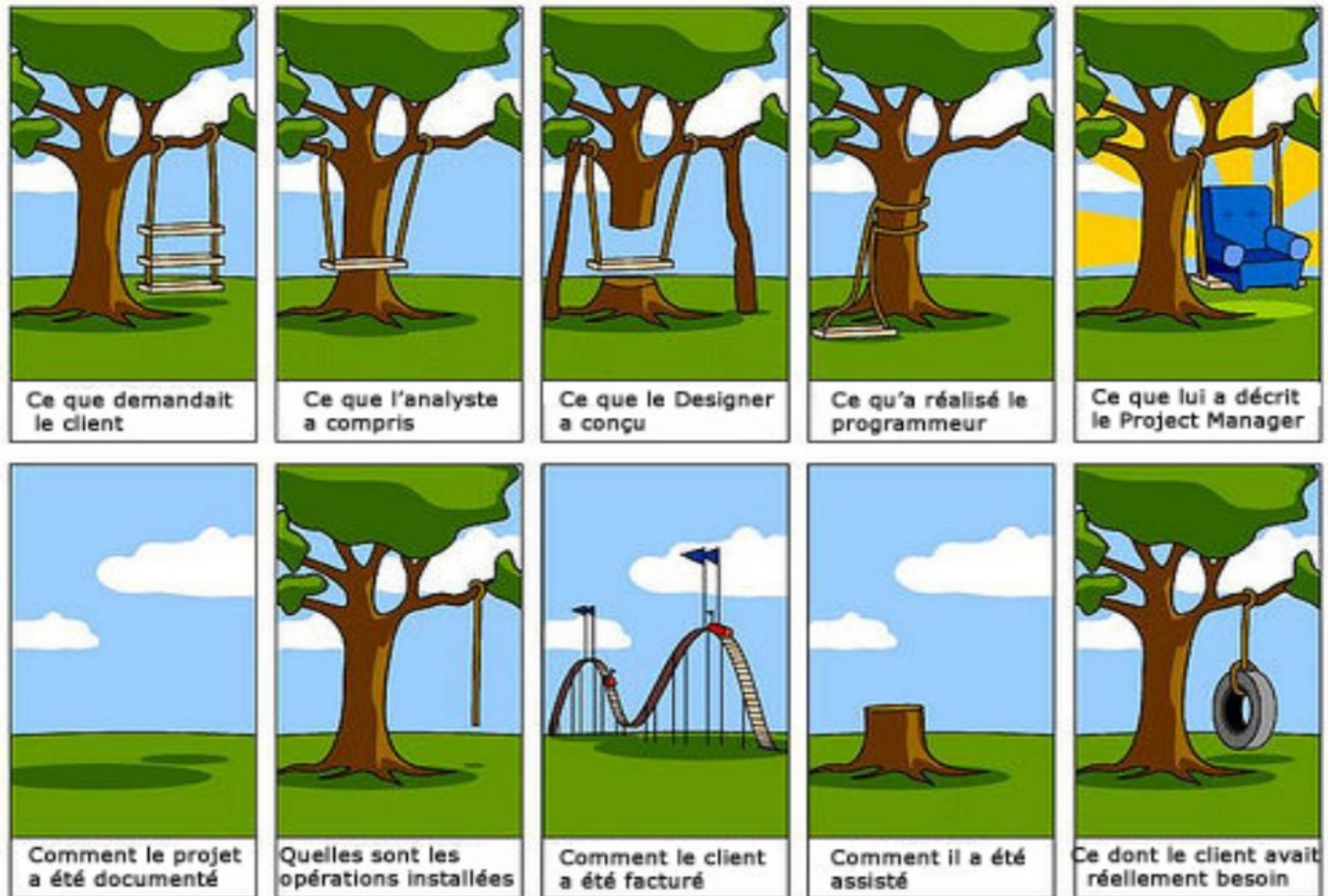
Activités : *Spécification*(*Bonnes pratiques*)

Une bonne définition des exigences est capitale et contribue à la réussite du projet en :

permettant d'obtenir un accord explicite entre les développeurs, les clients et les utilisateurs sur le travail à réaliser et les critères d'acceptation du système final,
fournissant une base solide pour l'estimation des ressources nécessaires (temps, coût, équipement, qualifications des développeurs,...),
évitant les incompréhensions et les omissions.



Activités : *Spécification*(Difficultés)



Activités : *Spécification*(*Cahier de Charges*)

- Les efforts de la spécification doit être consigné dans un **cahier de charges (CDC)**.
- Le cahier des charges est un document recensant les spécifications/exigences. Il résulte de l'ana-
- lyse et est contractuel entre le client et l'entreprise informatique qui va réaliser le logiciel. Il doit
- donc être validé par les deux. Qualités attendues : non ambigu, complet, vérifiable, cohérent, modifiable, traçable, utilisable durant la maintenance, indépendant des solutions (voir notamment la norme IEEE 830).

Activités : *Spécification(Cahier de Charges)*

- Les efforts de la spécification doit être consigné dans un **cahier de charges (CDC)**.
- Le cahier des charges est un document recensant les spécifications/exigences. Il résulte d'un dialogue et est contractuel entre le client et l'entreprise informatique qui va réaliser le logiciel.
- Il doit donc être validé par les deux.
 - Qualités attendues :
 - non ambigu, complet, vérifiable, cohérent, modifiable, traçable, utilisable durant la maintenance, indépendant des solutions (voir notamment la norme IEEE 830).

Activités : *Spécification*(Cahier de Charges)

•Plan type :

•1. introduction : présentation générale, motivations, définitions des termes

•2. contexte : environnement matériel et humain, acteurs et utilisateurs, interaction avec

•d'autres systèmes et logiciels, existant

•3. spécifications fonctionnelles : grandes fonctionnalités du système, acteurs et autres systèmes qu'elles impliquent

•4. spécifications non fonctionnelles, contraintes :

- charte graphique
- matériel : marques, RAM, débit de connexion Internet...
- interfaçage : protocoles de communication, formats de fichiers, etc., pour l'interaction avec des matériels, logiciels, systèmes d'exploitation...
- performances : temps réel...
- sécurité : sauvegardes, confidentialité, ...
- charge à supporter : volume des données, nombre d'utilisateurs simultanés, ...
- comportement en cas de panne...

•5. Liste des ressources Matérielles, Logicielles et Humaine.

•6. Planification (eg. GANTT, PERT)

•7. priorités relatives des spécifications, versions à prévoir, délais

•8. évolutions à prévoir

•9. annexes

•Couramment, les points 2 à 6 sont présentés en deux temps, de façon générale puis de façon

•détaillée, donnant alors lieu à deux parties organisées essentiellement de la même façon. Une exigence

•générale se décline alors en plusieurs exigences détaillées.

Activités : *Spécification(Cahier de Charges)*

- Qualités requises dans un cahier de charges :
 - Bon niveau de généralité
 - Formulation adéquate des besoins ? Problème bien décrit
 - Être précis, non ambigu malgré l'usage d'un langage naturel (/= mathématique)
 - Être complet (pas d'omission involontaire)
 - Être cohérent (pas d'inférence de fonctionnalités)
 - Être vérifiable : critères de validation définis. Évaluer la faisabilité des besoins ? faire éventuellement une maquette, une simulation
 - Être modifiable : facilité à exprimer un changement ou ajout de besoins

Activités : *Spécification(Cahier de Charges)*

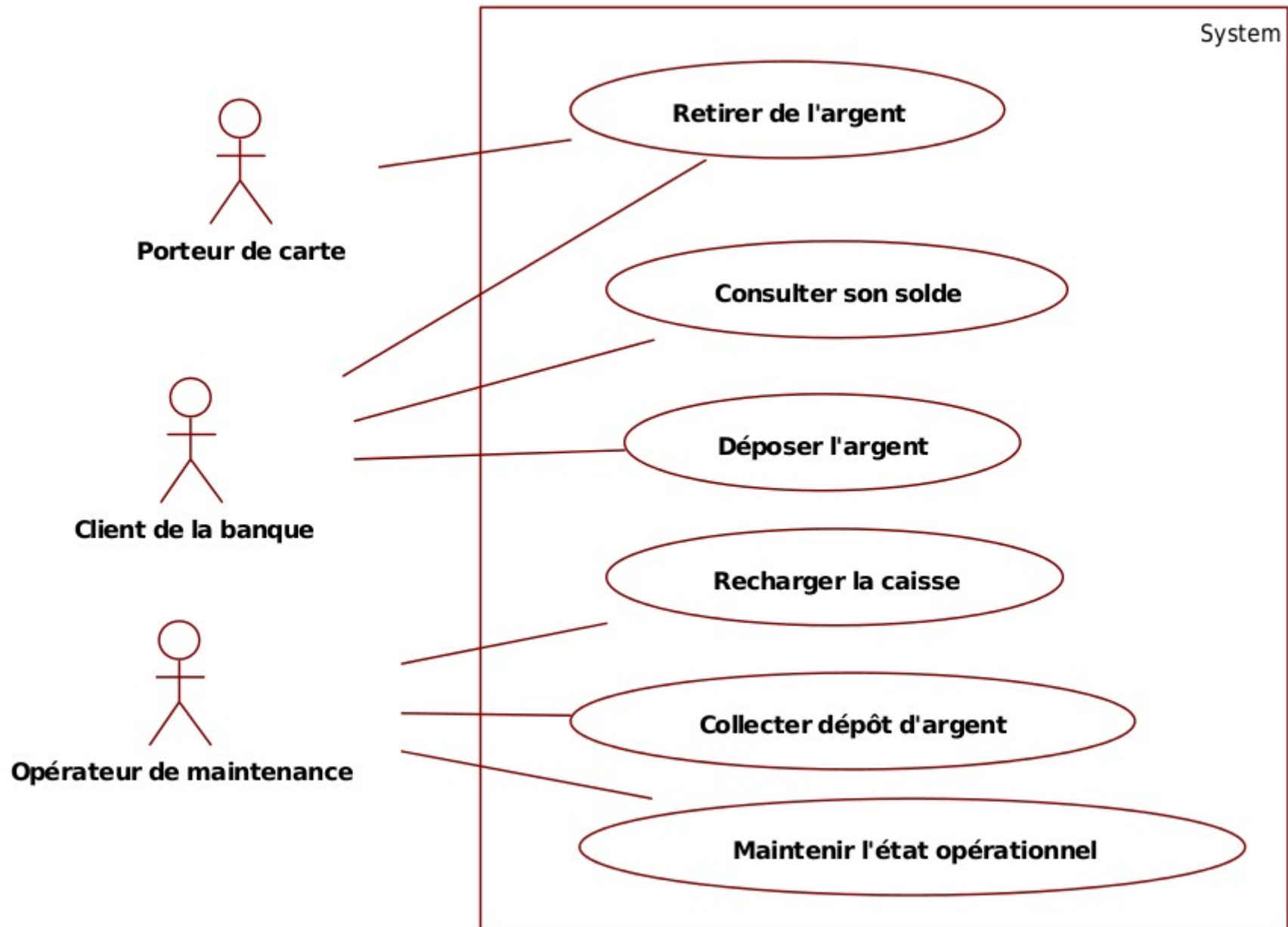
- Rédaction d'un cahier des charges :
 - norme IEEE 830 (1998) pour la rédaction des spécifications ;organisation : numérotation, tableaux, schémas, exemples. . . (ce n'est pas un roman !) ;
 - diagrammes UML : en particulier des cas d'utilisation pour présenter les fonctionnalités et les acteurs qu'elles impliquent ;
 - autres schémas : écrans types, diagramme de déploiement UML...
 - scénarios (d'interaction) : illustration des cas d'utilisation complexes par un exemple d'interaction le réalisant ; spécifier les acteurs impliqués et l'enchaînement des interactions sur un exemple, ainsi que le traitement des erreurs et les éventuelles préconditions et postconditions ;

Activités : *Spécification(Exemple)*

Un Guichet Automatique de Banque (GAB) offre les services suivants :

- Distribution d'argent à tout Porteur de carte de crédit, via un lecteur de carte et un distributeur de billets.
- Consultation de solde de compte, dépôt en numéraire et dépôt de chèques pour les clients porteurs d'une carte de crédit de la banque.
- Un opérateur de maintenance se charge de la collecte des dépôts d'argent et de la recharge du distributeur.

Activités : *Spécification(Exemple)*



Activités : *Spécification(Exemple)*

Besoins d'interface Homme/Machine (IHM)

- Les dispositifs d'entrée/sortie à la disposition du Porteur de carte doivent être :
 - Un lecteur de carte bancaire.
 - Un clavier numérique (pour saisir son code), avec des touches "validation", "correction" et "annulation".
 - Un écran pour l'affichage des messages du GAB.
 - Des touches autour de l'écran pour sélectionner un montant de retrait parmi ceux qui sont proposés.
 - Un distributeur de billets.
 - Un distributeur de tickets.

Activités : Conception(**Comment ?**)

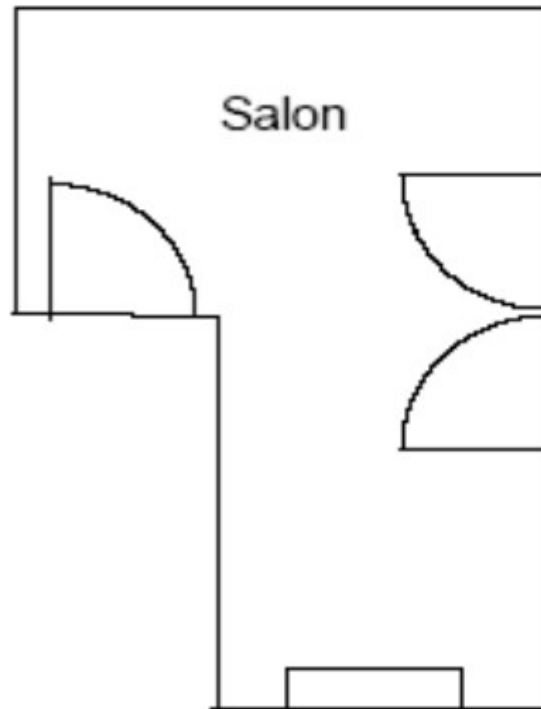
Activité	Concevoir
Description	Organiser le logiciel afin qu'il puisse satisfaire les exigences de la spécification. Faire les principaux choix techniques pour satisfaire les exigences de la spécification.
Entrées	Une spécification.
Sorties	Une description des décisions de conception. Des procédures de tests qui permettent de vérifier que les décisions de conception sont correctement implémentées en code source et qu'elles contribuent à satisfaire les exigences de la spécification.

Activités : Conception(*architecture*)

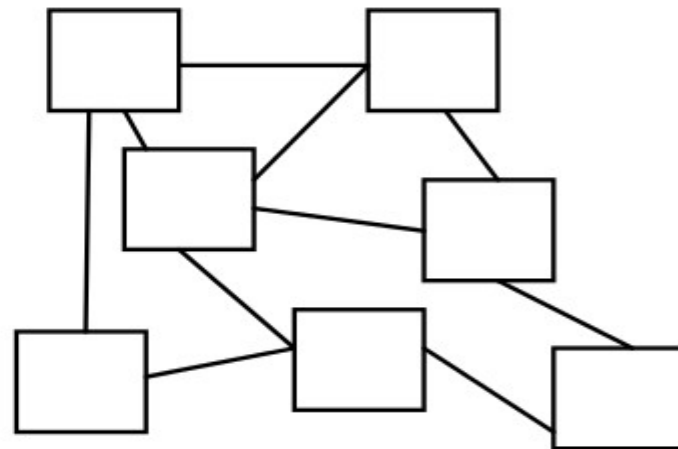
Décomposer le système en sous-systèmes

Définir les interfaces, les liens entre les composants

Résultat : Une description de l'architecture du logiciel.



plan général



Activités : Conception(*détaillée*)

- Détailler le fonctionnement des composants
- Définir quelques algorithmes, la représentation des données, ... des tests unitaires sont définis pour s'assurer que les composants réalisés sont bien conformes à leurs descriptions.
- Résultat : pour chaque composant, le résultat consiste en
 - Un dossier de conception détaillée.
 - Un dossier de tests unitaires.
 - Un dossier de définition d'intégration logiciel.

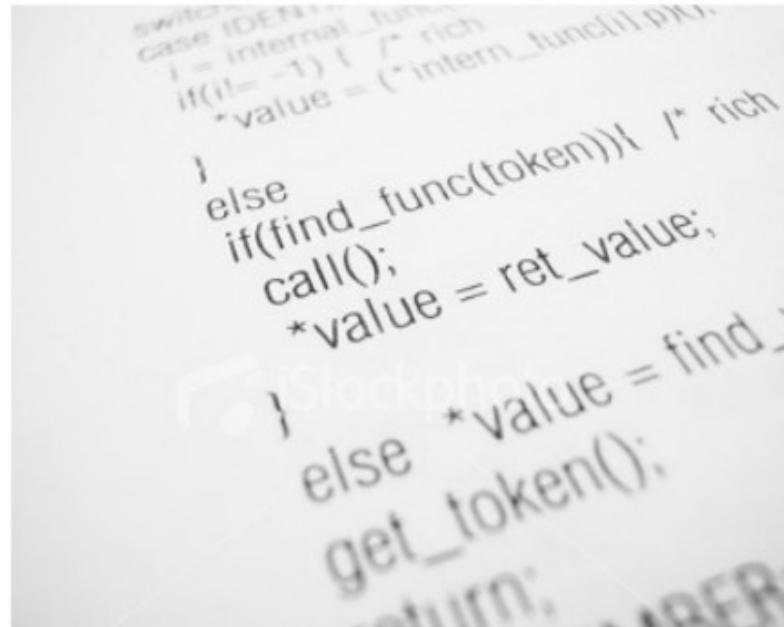
Activités : Conception

Qualité d'une conception

- La composante la plus importante de la qualité d'une conception est la maintenabilité.
 - maximiser la cohésion à l'intérieur des composants
 - minimiser le couplage entre ces composants

Activités : Implantation

Activité	Coder et tester
Description	Écrire le code source du logiciel. Tester le comportement du code source afin de vérifier qu'il réalise les responsabilités qui lui sont allouées.
Entrées	Spécification, conception.
Sorties	Code source. Tests unitaires. Documentation



Activités : Intégration

Activité	Intégrer
Description	Assembler le code source du logiciel (éventuellement partiellement) et dérouler les tests d'intégration.
Entrées	Conception, code source, tests d'intégration (tests).
Sorties	Un rapport de test d'intégration.



Activités : Validation

Activité	Valider
Description	Construire le logiciel complet exécutable. Dérouler les tests de validation sur le logiciel complet exécutable.
Entrées	Logiciel complet exécutable à valider. Tests de validation.
Sorties	Rapport de tests de validation.

Qualification (ou **recette**), c'est-à-dire la vérification de la conformité du logiciel aux spécifications initiales.



Activities: Maintenance

- **Types :**

- maintenance **corrective** : correction des bugs
- maintenance **adaptative** : ajuster le logiciel
- Maintenance **perfective**, d'extension :
augmenter/améliorer les possibilités du logiciel

- **Productions :**

- logiciel corrigé
- mises à jour
- documents corrigés

Maintenance corrective

- Corriger les erreurs : défauts d'utilité, d'utilisabilité, de fiabilité... Identifier la défaillance, le fonctionnement
Localiser la partie du code responsable
- Attention
 - La plupart des corrections introduisent de nouvelles erreurs
 - Les coûts de correction augmentent exponentiellement avec le délai de détection
 - Corriger et estimer l'impact d'une modification
- La maintenance corrective donne lieu à de nouvelles livraisons (release)

Maintenance adaptative

- Ajuster le logiciel pour qu'il continue à remplir son rôle compte tenu de l'évolution des
 - Environnements d'exécution
 - Fonctions à satisfaire
 - Conditions d'utilisation
- Ex : changement de SGBD, de machine, de taux de TVA, an 2000, euro...

Maintenance perfective

- **Maintenance perfective, d'extension**
 - Accroître/améliorer les possibilités du logiciel
 - Ex : les services offerts, l'interface utilisateur, les performances...
 - Donne lieu à de nouvelles versions.

Processus de développement : Pourquoi ?

- Un logiciel à développer est un **problème très complexe à résoudre**.
- Pour appréhender la complexité de développement du logiciel, il faut arriver :
 - à rendre les choses **simples. séparer les problèmes** qui peuvent l'être afin de les **résoudre de manière presque indépendante**. « Diviser pour mieux régner. »

Processus logiciel